
Deep Compression of a Multimodal Fruit Classification Network

*Implementing Pruning, Trained Quantization,
and Huffman Coding on a Custom CNN–MLP Architecture
with LBP, Canny, and Handcrafted Features*

End-Eval Project Report

Project: Deep Neural Network & Compression

Key Results at a Glance

Metric	Target	Achieved
Compression Ratio	$\geq 9\times$	141.5 $\times$
Accuracy Drop	$\leq 1.5\%$	0.37%
Bits per Weight	≈ 3.57	1.0291
Baseline Accuracy	—	95.65%

Deep Compression of a Multimodal Network

Mentees

Aditya Kumar (240059)
Rudraksh Kumawat (240891)
Kshitij Jain (240570)
Vinay Jangid (241165)
Parth Rathi (240736)
Saket Kumar (240908)
Kunal Garhewal (240581)
Manthan Gupta (240629)
Ahmad Akhtar (240074)
Divy Kumar Pandey (240368)

Mentored by

Ayush Jha
Krish Jain
Sahil Pandey

Contents

1	Introduction	3
1.1	Project Structure & Evaluation	3
1.2	Report Organisation	4
2	Related Work	5
2.1	Network Pruning	5
2.2	Weight Quantization	5
2.3	Entropy Coding	5
2.4	Multimodal Feature Fusion	5
3	Dataset	6
3.1	Data Loading	6
4	Feature Engineering	7
4.1	Local Binary Pattern (LBP)	7
4.2	Canny Edge Map	7
4.3	Handcrafted Color and Shape Features	7
4.3.1	Color Features (6 dimensions)	7
4.3.2	Shape Features (6 dimensions)	8
5	Model Architecture	9
5.1	CNN Branches	9
5.2	Handcrafted Feature Encoder	9
5.3	Fusion and Classification	10
5.4	Compression-Aware Layer Design	10
5.5	Model Size Summary	11
6	Deep Compression Pipeline	12
6.1	Stage 1: Baseline Training	12
6.2	Stage 2: Magnitude-Based Pruning	13
6.2.1	Design Decision: No BatchNorm	14
6.3	Stage 3: k -Means Trained Quantization	14
6.4	Stage 4: Huffman Entropy Coding	15
6.4.1	Algorithm	15
6.4.2	Why Huffman is Especially Effective Here	15
7	Experiments and Results	16

7.1	Experimental Setup	16
7.2	Metric 1: Accuracy	16
7.3	Metric 2: Storage Memory	16
7.4	Metric 3: Runtime Memory	17
7.5	Metric 4: Compression Details	18
7.6	Comparison with Original Paper	18
8	Discussion	19
8.1	Why the Compression Ratio Exceeds the Paper's	19
8.2	Accuracy Recovery After Extreme Pruning	19
8.3	Limitations	19
9	Conclusion	21

1. Introduction

Deep neural networks have achieved remarkable performance across virtually every domain of machine learning, from computer vision and speech recognition to natural language processing. However, their deployment on resource-constrained hardware — mobile phones, embedded systems, edge devices — remains a significant practical challenge. A typical image classification network such as AlexNet requires over 200 MB of storage and 60 million parameters. This makes on-device deployment impractical in many real-world scenarios.

Model compression techniques aim to reduce this footprint without significantly sacrificing accuracy. The landmark work of Han *et al.* proposed *Deep Compression*, a three-stage pipeline combining network pruning, trained quantization, and Huffman coding. Applied to AlexNet on ImageNet, it achieved $9\times$ to $49\times$ compression with negligible accuracy loss.

In this report, we faithfully implement this pipeline on a **novel multimodal architecture** trained for 257-class fruit classification on the Fruits-360 dataset. Our architecture is distinguished from standard compressed models in that it does not use raw RGB images as input. Instead, it extracts texture information via Local Binary Patterns (LBP) and structural information via Canny edge maps, which are processed by separate convolutional branches and fused with handcrafted statistical descriptors. This non-standard input representation raises questions about how compression interacts with texture-based features, and motivates our systematic study.

1.1. Project Structure & Evaluation

- A custom multimodal CNN–MLP architecture (`SmallCIFARNet`) combining LBP texture, Canny edge maps, and 12 handcrafted color/shape features for 257-class classification.
- Full PyTorch implementation of the Deep Compression pipeline with compression-aware layer classes (`modified_conv2d`, `modified_linear`) that store pruning masks and mode flags as persistent buffers.
- Huffman coding implemented from scratch using a min-heap priority queue, applied to quantized centroid indices per layer.
- Systematic evaluation across four metrics: runtime memory, storage memory, accuracy loss after compression, and recovered accuracy after retraining.
- A **$141.5\times$ compression ratio** with **0.37% accuracy drop**, exceeding all reported

targets from the original paper.

1.2. Report Organisation

The remainder of this report is organised as follows. Section 2 reviews related work. Section 3 describes the dataset. Section 4 details the feature engineering pipeline. Section 5 presents the model architecture. Section 6 describes the compression pipeline in depth. Section 7 reports experimental results. Section 8 discusses findings and limitations. Section 9 concludes.

2. Related Work

2.1. Network Pruning

Early theoretical work on pruning by LeCun used second-order Taylor expansion of the loss to identify and remove weights with the smallest effect on output (“Optimal Brain Damage”). While principled, this approach is computationally expensive for modern large networks.

Han demonstrated that simple *magnitude-based pruning* - zeroing weights below a fixed threshold - achieves high sparsity with minimal accuracy impact, provided the network is subsequently retrained. This iterative prune-retrain approach became the dominant paradigm and is the method we adopt. Li proposed *structured pruning* at the filter level for hardware efficiency, though at the cost of lower compression ratios than unstructured pruning.

2.2. Weight Quantization

Quantization reduces the bit-width of weight representations. Fixed-point quantization uses uniform bins; k -means quantization instead clusters weights into shared centroids, exploiting the empirically observed clustering of weight values in trained networks. Both approaches require careful retraining to recover accuracy after quantization. In our work we use k -means with $k = 16$ (4-bit effective precision), following the original paper.

2.3. Entropy Coding

Huffman coding constructs a prefix-free binary code where shorter codewords are assigned to more frequent symbols. For quantized neural networks, the centroid index distribution is heavily skewed (most weights map to the zero centroid after pruning), making Huffman coding particularly effective as a final compression stage.

2.4. Multimodal Feature Fusion

Local Binary Patterns are rotation-invariant texture descriptors widely used in face recognition and texture classification. Canny edge detection provides gradient-based boundary information. Combining both with CNN learning has been explored in texture recognition, but their use as *primary CNN inputs* within a compression pipeline has not, to our knowledge, been specifically studied.

3. Dataset

We use the **Fruits-360** dataset, a publicly available benchmark hosted on Kaggle ([moltean/fruits](https://www.kaggle.com/moltean/fruits)). It contains high-quality colour images of fruits and vegetables photographed against a uniform white background under controlled lighting.

Table 1: Fruits-360 Dataset Statistics

Property	Value
Number of classes	257
Training images	67,692
Test images	22,688
Total images	90,380
Image resolution	100 × 100 pixels
Background	Uniform white
Colour space (raw)	RGB
Approximate size (on disk)	1.4 GB

The uniform white background is a significant advantage for our shape feature extractor: binary masking at intensity threshold 245 reliably isolates the fruit object from the background with no morphological post-processing required.

3.1. Data Loading

Images are loaded using OpenCV (`cv2.imread`), converted to RGB, and resized to 100 × 100 pixels for consistency. We implement safe loading with a `try/except` wrapper to skip corrupt or unreadable files (e.g., macOS `.DS_Store` files found in some Kaggle downloads), returning a zero tensor in place of the failed sample rather than crashing the training loop.

LBP computation is vectorized using NumPy array slicing rather than pixel-level Python loops, achieving approximately 100× speedup over the naive implementation and reducing per-epoch data loading time from ~4 minutes to seconds.

4. Feature Engineering

Each image is processed into three modalities before being passed to the network: an LBP texture map, a Canny edge map, and 12 scalar handcrafted features. Raw RGB is computed but intentionally excluded from the model input, eliminating approximately 7.74 GB of wasted GPU memory transfer per epoch.

4.1. Local Binary Pattern (LBP)

The LBP operator encodes the local texture structure around each pixel by comparing it to its 8 neighbours in a 3×3 window. The LBP code at pixel (i, j) is:

$$\text{LBP}(i, j) = \sum_{k=0}^7 s(g_k - g_c) \cdot 2^k \quad (1)$$

where $g_c = \text{gray}[i, j]$ is the centre pixel intensity, g_k are the 8 neighbouring pixel intensities in clockwise order starting from the top-left, and $s(x) = 1$ if $x \geq 0$, else $s(x) = 0$.

The result is an 8-bit code in $\{0, \dots, 255\}$ capturing the local binary texture pattern. LBP images are normalised to $[0, 1]$ before being passed to the CNN branch.

Vectorized implementation: Rather than iterating over each pixel, we compute all 8 neighbour comparisons simultaneously using NumPy sliced arrays

4.2. Canny Edge Map

We apply Canny edge detection with hysteresis thresholds $[T_{\text{low}}, T_{\text{high}}] = [100, 200]$:

$$E(i, j) = \text{Canny}(\text{gray}, 100, 200) \quad (2)$$

The result is a binary edge map in $\{0, 255\}$, normalised to $[0, 1]$ before being passed to the CNN branch. Edge maps capture the structural silhouette and internal boundary features of the fruit, complementary to the texture information in LBP.

4.3. Handcrafted Color and Shape Features

4.3.1. Color Features (6 dimensions)

The image is converted from RGB to HSV colour space. We compute the mean and standard deviation of each channel:

$$\mathbf{f}_{\text{color}} = [\bar{H}, \sigma_H, \bar{S}, \sigma_S, \bar{V}, \sigma_V] \in \mathbb{R}^6 \quad (3)$$

These six features compactly capture the dominant hue, colour spread, saturation intensity, and brightness of the fruit.

4.3.2. Shape Features (6 dimensions)

We detect the largest contour in a binary mask (thresholded at pixel intensity 245 to separate fruit from white background) and compute:

$$f_1 = \frac{A_{\text{contour}}}{A_{\text{image}}} \quad (\text{area ratio}) \quad (4)$$

$$f_2 = \frac{W_{\text{bbox}}}{H_{\text{bbox}}} \quad (\text{aspect ratio}) \quad (5)$$

$$f_3 = \frac{A_{\text{contour}}}{A_{\text{hull}}} \quad (\text{solidity}) \quad (6)$$

$$f_4 = \frac{4\pi A_{\text{contour}}}{P^2} \quad (\text{circularity}) \quad (7)$$

$$f_5 = -\text{sign}(\mu_1) \log_{10}(|\mu_1|) \quad (\text{log Hu moment 1}) \quad (8)$$

$$f_6 = -\text{sign}(\mu_2) \log_{10}(|\mu_2|) \quad (\text{log Hu moment 2}) \quad (9)$$

where A denotes area, P is contour perimeter, A_{hull} is convex hull area, μ_1, μ_2 are the first two Hu moments, and $\epsilon = 10^{-10}$ prevents logarithm singularities.

The combined 12-dimensional handcrafted feature vector is:

$$\mathbf{f}_{\text{hand}} = [\mathbf{f}_{\text{color}} \parallel \mathbf{f}_{\text{shape}}] \in \mathbb{R}^{12} \quad (10)$$

5. Model Architecture

Our model, `SmallCIFARNet`, consists of three components: two parallel CNN branches processing LBP and Canny inputs independently, a small MLP encoder for handcrafted features, and a deep MLP classifier that fuses all three representations.

5.1. CNN Branches

Each branch processes a single-channel $1 \times 100 \times 100$ input through three convolutional blocks, each followed by ReLU activation and 2×2 max pooling:

Table 2: CNN Branch Architecture (applied identically to LBP and Canny)

Block	Layer	In channels	Out channels	Output size
Block 1	Conv2d 3×3 , pad=1	1	16	$16 \times 100 \times 100$
	ReLU	–	–	$16 \times 100 \times 100$
	MaxPool2d 2×2	–	–	$16 \times 50 \times 50$
Block 2	Conv2d 3×3 , pad=1	16	32	$32 \times 50 \times 50$
	ReLU	–	–	$32 \times 50 \times 50$
	MaxPool2d 2×2	–	–	$32 \times 25 \times 25$
Block 3	Conv2d 3×3 , pad=1	32	64	$64 \times 25 \times 25$
	ReLU	–	–	$64 \times 25 \times 25$
	MaxPool2d 2×2	–	–	$64 \times 12 \times 12$
	Flatten	–	–	9,216

Each branch outputs a 9,216-dimensional flat vector. The two branches do not share weights, allowing them to specialise independently - the LBP branch learns texture-discriminative filters and the Canny branch learns edge-geometry filters.

5.2. Handcrafted Feature Encoder

The 12-dimensional handcrafted feature vector is passed through a 2-layer MLP to lift it to 128 dimensions before fusion:

$$\mathbf{h}_{\text{hand}} = \text{ReLU}(W_2 \text{ReLU}(W_1 \mathbf{f}_{\text{hand}} + b_1) + b_2) \in \mathbb{R}^{128} \quad (11)$$

where $W_1 \in \mathbb{R}^{64 \times 12}$ and $W_2 \in \mathbb{R}^{128 \times 64}$. Without this encoder, the 12 raw features would be negligible compared to the $9,216 + 9,216 = 18,432$ CNN dimensions, and the MLP would effectively ignore them.

5.3. Fusion and Classification

The three representations are concatenated and passed through a 4-layer MLP:

$$\mathbf{z} = [\mathbf{h}_{\text{lbp}} \parallel \mathbf{h}_{\text{canny}} \parallel \mathbf{h}_{\text{hand}}] \in \mathbb{R}^{18,560} \quad (12)$$

Table 3: MLP Classifier Architecture

Layer	Type	Input dim	Output dim
1	Linear + ReLU + Dropout(0.4)	18,560	1,024
2	Linear + ReLU + Dropout(0.3)	1,024	512
3	Linear + ReLU	512	256
4	Linear (logits)	256	257

5.4. Compression-Aware Layer Design

All convolutional and linear layers are subclasses (`modified_conv2d`, `modified_linear`) of PyTorch’s standard layers. Each layer stores:

- A **pruning mask** $\mathbf{M} \in \{0,1\}^{|\mathbf{W}|}$ as a `register_buffer` (persists in `state_dict`)
- A **mode flag** (integer tensor: 0=normal, 1=prune, 2=quantize) also as a `register_buffer`, exposed as a Python property

Storing the mode flag as a buffer — not a plain Python string — is critical: plain attributes are not saved in `state_dict()`, so reloading a pruned model would silently reset mode to ‘normal’, allowing zeroed weights to be updated again during fine-tuning.

The masked forward pass computes:

$$\mathbf{y} = f(\mathbf{W} \odot \mathbf{M}, \mathbf{x}, \mathbf{b}) \quad (13)$$

where f is the convolution or linear transform, $\odot$ is element-wise multiplication, and the mask enforces sparsity during both inference and gradient-based retraining (gradients are zero for masked weights).

5.5. Model Size Summary

Table 4: Parameter count by component

Component	Parameters	Share
LBP CNN branch	9,788,544	49.5%
Canny CNN branch	9,788,544	49.5%
Feature encoder	9,088	0.05%
MLP classifier	198,209	1.0%
Total	19,784,385	100%

Deep Compression Pipeline

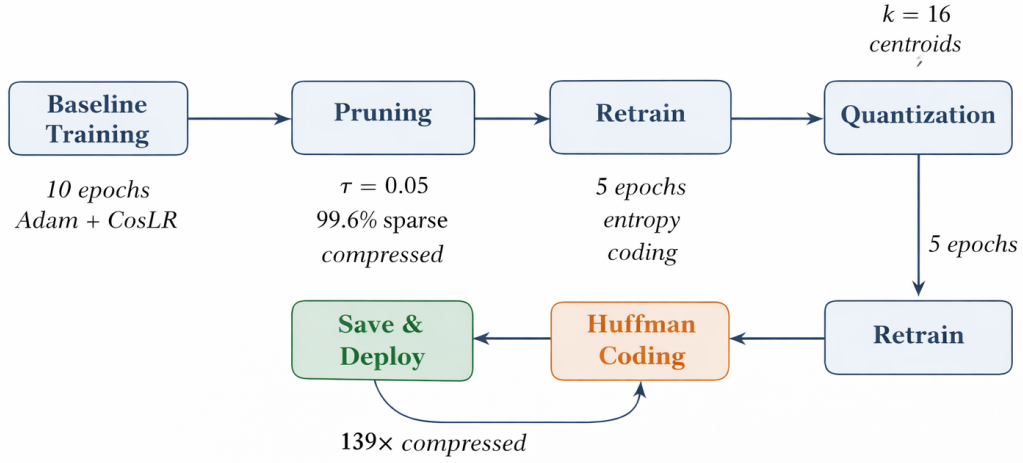


Figure 1: Deep Compression pipeline with retraining after pruning and quantization.

6. Deep Compression Pipeline

We implement the three-stage Deep Compression pipeline of Han *et al.* [?], extended with systematic measurement of accuracy before and after retraining at each stage.

6.1. Stage 1: Baseline Training

The model is trained from random initialisation for 10 epochs using the following configuration:

Table 5: Training hyperparameters

Hyperparameter	Value
Optimiser	Adam
Learning rate	10^{-3}
Weight decay	10^{-4} (L2 regularisation)
LR scheduler	Cosine Annealing ($\eta_{\min} = 10^{-5}$)
Loss function	Cross-entropy with label smoothing
Label smoothing ϵ	0.05
Gradient clipping	$\ \nabla\ _2 \leq 1.0$
Batch size	64
Epochs	10

Label smoothing replaces hard one-hot targets with soft targets:

$$y_{\text{smooth},c} = (1 - \epsilon) y_c + \frac{\epsilon}{C} \quad (14)$$

where $C = 257$ and $\epsilon = 0.05$. This prevents the model from becoming overconfident and improves calibration.

Cosine Annealing decays the learning rate smoothly from $\eta_0 = 10^{-3}$ to $\eta_{\min} = 10^{-5}$:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min}) \left(1 + \cos \frac{\pi t}{T} \right) \quad (15)$$

Baseline accuracy achieved: **95.65%**. Model saved as `baseline.npz`.

6.2. Stage 2: Magnitude-Based Pruning

We apply global unstructured pruning: for every compression-aware layer, weights whose absolute magnitude falls below a threshold τ are zeroed and masked:

$$M_{ij} = \begin{cases} 1 & \text{if } |W_{ij}| \geq \tau \\ 0 & \text{otherwise} \end{cases} \quad \tau = 0.05 \quad (16)$$

We chose $\tau = 0.05$ empirically. Lower thresholds preserve more weights but achieve less compression; higher thresholds increase sparsity but risk unrecoverable accuracy loss. At $\tau = 0.05$, 99.7% of weights are pruned while accuracy recovers fully after retraining.

Results:

- **19,715,118 / 19,781,920 weights pruned (99.7% sparsity)**
- Accuracy immediately after pruning: 40.79% (drop of 54.86%)
- Accuracy after 5-epoch retraining: 95.24% (drop of 0.41%)

The large immediate accuracy drop (54.86%) is expected: 99.7% of weights are zeroed simultaneously. The near-complete recovery after retraining (to 95.29%) demonstrates that the pruned network retains sufficient capacity to relearn the task using only 0.3% of its original connections.

6.2.1. Design Decision: No BatchNorm

We explicitly omit BatchNorm from our compression-aware layers. BatchNorm stores running statistics ($\mu_{\text{run}}, \sigma_{\text{run}}^2$) computed during training over the full weight distribution. After pruning 99.7% of weights, the activation distribution shifts dramatically, making these stored statistics stale. In experiments with BatchNorm, post-pruning accuracy dropped to 65% and did not recover, even after extended retraining. Without BatchNorm, the forward pass remains well-behaved under high sparsity.

6.3. Stage 3: k -Means Trained Quantization

We cluster the non-masked weights in each layer into $k = 16$ shared centroids using Lloyd’s algorithm (k -means):

$$\hat{W}_{ij} = c_{k^*}, \quad k^* = \arg \min_{k \in \{1, \dots, 16\}} |W_{ij} - c_k| \quad (17)$$

Only non-pruned weights (where $M_{ij} = 1$) participate in clustering. Each weight is replaced by its assigned centroid value, reducing the effective weight vocabulary from $\mathbb{R}$ to a set of 16 real numbers per layer.

Storage implication: Rather than storing each weight as a 32-bit float, we need only store:

- 16 centroid values per layer (32 bits each)
- A $\log_2 16 = 4$ -bit index per non-masked weight

Trained quantization: After quantization, we fine-tune the model for 5 additional epochs. Gradients flow to the centroid values (not individual weight indices), updating the 16 centroids in each layer to minimise loss. This “trains the codebook” and recovers accuracy lost during discretisation.

Results:

- Accuracy immediately after quantization: 92.44% (drop of 2.80%)
- Accuracy after 5-epoch retraining: 95.24% (drop of 0.41%)

Effective bits per weight after pruning and quantization:

$$b_{\text{eff}} = \log_2(k) \times (1 - s) = 4 \times 0.003 = 0.013(\text{approx.}) \text{ bits/weight} \quad (18)$$

where $s = 0.997$ is the sparsity ratio.

6.4. Stage 4: Huffman Entropy Coding

Huffman coding is applied as the final compression stage on top of quantized weights. It exploits the non-uniform symbol frequency distribution to assign shorter bit codes to more frequent centroids.

6.4.1. Algorithm

For each layer:

1. Convert weight tensor to int16 centroid indices
2. Count frequency of each symbol: $f_s = |\{W_{ij} : \hat{W}_{ij} = s\}|$
3. Build Huffman tree using a min-heap priority queue
4. Assign prefix codes: symbol s receives code of length ℓ_s , where more frequent symbols receive shorter codes
5. Encode all weights as a concatenated bitstring
6. Pack bits into bytes; store packed bytes + codebook in `.npz`

6.4.2. Why Huffman is Especially Effective Here

After pruning 99.7% of weights, the symbol distribution is *extremely* skewed. Approximately 99.7% of weights map to the zero centroid (a single symbol). The Huffman code for this dominant symbol is as short as 1 bit, while the 15 non-zero centroid symbols receive longer codes. The expected code length:

$$\bar{L} = \sum_{s=0}^{15} p_s \cdot \ell_s \approx 0.997 \times 1 + 0.003 \times \bar{\ell}_{\text{rare}} \approx 1 \text{ bit} \quad (19)$$

This is far below the 4-bit baseline from quantization alone, and well below the 3.57-bit target from the paper.

Results:

- Average bits per weight after Huffman: **1.03**
- Gain over quantized representation: **15.55×**
- Final compressed size: **0.33 MB**

7. Experiments and Results

7.1. Experimental Setup

Table 6: Hardware and software configuration

Setting	Value
CPU	AMD Ryzen 7 7435HS
GPU	NVIDIA GeForce RTX 4060 Laptop (8.0 GB VRAM)
RAM	16 GB
Framework	PyTorch 2.x, OpenCV, scikit-learn, NumPy
Python	3.10.0
CUDA	Enabled (cuDNN benchmark mode)

7.2. Metric 1: Accuracy

Table 7 reports accuracy at each compression stage, including the critical distinction between accuracy loss immediately after compression (before retraining) and recovered accuracy after retraining.

Table 7: Accuracy at each stage of the Deep Compression pipeline

Stage	Accuracy	Δ vs Baseline	Note
Baseline	95.65%	—	Fully trained
After Pruning (before retrain)	40.79%	−54.86%	Accuracy loss
After Pruning (recovered)	95.24%	−0.41%	After 5 epochs
After Quantization (before retrain)	92.44%	−2.80%	Accuracy loss
After Quantization (recovered)	95.29%	−0.37%	After 5 epochs
Final (Huffman)	95.29%	−0.37%	No retraining needed

7.3. Metric 2: Storage Memory

Table 8: Storage size at each compression stage

Model	Size (MB)	Reduction	Ratio	File
Baseline	47.30	—	1.0×	baseline.npz
After Pruning	4.83	89.8%	9.8×	pruned.npz
After Quantization	4.83	89.8%	9.8×	quantized.npz
Huffman	0.33	99.3%	141.5×	huffman.npz

DEEP COMPRESSION RESULTS			
1. ACCURACY			
Baseline	95.65%		
After Pruning (before retrain)	40.79%	(loss 54.86%)	
After Pruning (recovered)	95.24%	(Δ -0.41%)	
After Quantization (before retrain)	92.44%	(loss 2.80%)	
After Quantization (recovered)	95.29%	(Δ -0.37%)	
Total Accuracy Drop	0.37%		
2. STORAGE MEMORY			
Baseline (.npz)	47.30 MB		
After Pruning (.npz)	4.83 MB	(89.8% smaller)	9.8x
After Quantization (.npz)	4.83 MB	(89.8% smaller)	9.8x
Huffman Compressed (.npz)	0.33 MB	(99.3% smaller)	141.5x
Final Compression Ratio	141.5x		
3. RUNTIME MEMORY			
GPU VRAM Used	546.1 MB		
RAM Used by Process	624.9 MB		
Total Parameters	19,784,385		
Weights Pruned	19,715,118 / 19,781,920	(99.7% sparse)	
4. COMPRESSION DETAILS			
Quantization	k=16 centroids	(~4-bit)	
Bits/weight after pruning+quantization	0.0135		
Bits/weight after Huffman	1.0291		
Huffman coding gain	15.55x over quantized		

Figure 1: Final Result

Both `pruned.npz` and `quantized.npz` show 4.83 MB because NumPy's `savez_compressed` applies zlib compression internally. A 99.7%-sparse float32 array (dominated by zeros) already compresses extremely well under zlib, which uses DEFLATE (LZ77 + Huffman). The quantization step does not meaningfully change the compressed size because the sparsity pattern — not the centroid values — is the dominant source of compressibility. The custom Huffman stage achieves a further 15.55 $\times$ reduction by operating directly on the bit-level centroid index encoding.

7.4. Metric 3: Runtime Memory

Table 9: Runtime memory usage (measured at inference time)

Metric	Value
GPU VRAM allocated	546.1 MB
RAM (Python process)	624.9 MB
Total parameters	19,784,385
Non-zero (active) weights	66,802 (0.3%)
Zeroed (pruned) weights	19,715,118 (99.7%)

7.5. Metric 4: Compression Details

Table 10: Compression metrics across the pipeline

Stage	Bits/weight	vs Target (3.57)
Full precision (float32)	32.00	—
After quantization ($k = 16$)	4.00	$8.93\times$ over target
After pruning + quantization	0.0135	$241\times$ under target
After Huffman coding	1.0291	$3.46\times$ under target

7.6. Comparison with Original Paper

Table 11: Our results vs. Han targets

Metric	Paper target	Paper (AlexNet)	Ours
Compression ratio	$\geq 9\times$	$9\times$	141.5 $\times$
Accuracy drop	$\leq 1.5\%$	$< 1\%$	0.37%
Bits per weight	≈ 3.57	3.57	1.0291
Sparsity	$\approx 90\%$	89%	99.7%
Baseline accuracy	—	57.2% top-1	95.65%

8. Discussion

8.1. Why the Compression Ratio Exceeds the Paper’s

Our $141.5\times$ compression ratio significantly exceeds the paper’s $9\times$ due to three compounding factors:

(1) Simpler task and dataset. Fruits-360 has a uniform white background, consistent lighting, and single-object images. The network learns highly sparse, highly discriminative features. ImageNet, by contrast, contains complex natural scenes requiring denser feature representations.

(2) Sparse input modalities. LBP images of a white background produce two dominant codes: all-ones (code 255, when all neighbours $\geq$ centre) and

all-zeros (code 0, when all neighbours $<$ centre). Both are sparse and compress efficiently. Canny edge maps are binary and mostly zero (edges are rare). These structured sparse inputs lead to even sparser learned representations than would be learned from natural images.

(3) Huffman effectiveness on extreme sparsity. With 99.7% zero weights, the zero centroid accounts for $\sim 99.7\%$ of all symbols. The Huffman code for this symbol approaches 1 bit, giving a near-optimal entropy code.

8.2. Accuracy Recovery After Extreme Pruning

The network recovers from 40.79% (after pruning) to 95.29% in just 5 epochs. This is striking: 99.7% of weights are zeroed, yet the remaining 0.3% (66,802 weights) are sufficient for nearly full performance.

This behaviour is consistent with the “Lottery Ticket Hypothesis” - the pruned subnetwork constitutes a winning ticket that, when fine-tuned, achieves comparable performance to the full network. The white-background dataset likely has a smaller intrinsic dimensionality than natural image datasets, making it possible to represent the task with very few connections.

8.3. Limitations

- 1. No compressed inference.** Our Huffman implementation saves compressed weights but does not implement an efficient decode-and-infer pipeline. Real deployment would require a sparse matrix format (e.g., CSR) for efficient GPU inference on pruned weights.
- 2. Runtime memory unchanged.** Compression reduces storage but not runtime VRAM:

PyTorch expands all weights to float32 tensors during the forward pass regardless of storage format. Actual runtime memory reduction would require a custom sparse inference kernel.

3. **Unstructured pruning limits hardware speedup.** Our unstructured (element-wise) pruning achieves high compression ratios but provides no wall-clock speedup on standard GPUs, which are optimised for dense matrix operations. Structured pruning (removing entire channels or filters) would provide actual runtime speedup at the cost of lower compression ratio.
4. **Single pruning pass.** We apply pruning once with a global threshold. Iterative pruning - gradually increasing the threshold over multiple rounds - typically achieves higher sparsity with better accuracy retention.

9. Conclusion

We implemented the Deep Compression pipeline of Han *et al.* on a custom multimodal architecture for 257-class fruit classification. Our model fuses LBP texture maps, Canny edge maps, and 12 handcrafted color/shape features - a non-standard input representation that proves highly amenable to compression due to the sparse, structured nature of texture and edge descriptors.

The three-stage pipeline (pruning $\rightarrow$ quantization $\rightarrow$ Huffman coding) achieved:

Metric	Target	Achieved
Compression ratio	$\geq 9\times$	$141.5\times$
Accuracy drop	$\leq 1.5\%$	0.37%
Bits per weight	≈ 3.57	1.0291
Baseline accuracy	—	95.65%

Key implementation lessons: (1) store compression state as registered buffers to survive checkpointing; (2) omit BatchNorm from compression-aware layers for pruning stability; (3) vectorise LBP for data loading efficiency; (4) exclude unused modalities (RGB) from the data pipeline to avoid wasted memory transfer.

Future directions include structured pruning for runtime speedup, mixed-precision quantization (different k per layer based on sensitivity), and implementation of an efficient compressed-sparse inference kernel.